DESKTOP UX GUIDE

HYPERCLUSTER DESKTOP ? UX GUIDE

Liquid Glass desktop UI for the hypercluster CLI. Every cluster action runs through the hypercluster binary ? the app never calls helm, kubectl, or SSH directly for lifecycle operations.
Launch: npm run desktop from repo root, or hypercluster-desktop after global install.

Table of Contents

1. Getting Started
2. Navigation
3. Dashboard
4. Create Cluster Wizard
5. Day-2 Operations
6. Deploy & Log Console
7. Settings
8. Command Palette
9. Keyboard Shortcuts
10. Notifications & Tray
11. Data Storage
12. CLI Mapping Reference

1. Getting Started

Install and run:

```
cd hypercluster
npm install
npm run desktop
```

Global install (optional):

```
npm install -g ./desktop
hypercluster-desktop    # alias: hc-desktop
```

First launch checklist:

1. Open **Settings** and confirm the resolved **hypercluster binary** path (repo ./hypercluster is auto-detected in dev).
2. **Import** an existing cluster.conf or use **Create Cluster** wizard.
3. Run **preflight** from Operations before first **deploy**.

Auto-import:

On startup, if the configured hypercluster binary lives in a repo that contains cluster.conf, the app imports that cluster once (when the cluster name is not already in the registry).

2. Navigation

The left sidebar has four views:

View · Purpose
Dashboard · Cluster cards, stats, import, quick deploy
Create Cluster · 5-step wizard ? cluster.conf
Operations · Day-2 commands for selected cluster
Settings · Binary path, import, data directory
The footer shows **?K command palette** hint (Ctrl+K on Windows/Linux).

3. Dashboard

Header:

- Shows resolved **hypercluster binary** path.
- **Import cluster.conf** ? native file picker; copies config into app registry.
- **+ New Cluster** ? opens the wizard.

Stat cards:

Card · Meaning
Clusters · Count in local registry
Ready nodes · Sum of ready nodes (status --json per cluster)
Active job · 1 while a hypercluster subprocess is running

Cluster cards:

Each card shows:

- Cluster **name** and **distro** (k3s or kubespray)
- **Status pill** ? unknown / ready / deploying (from last action)
- **Nodes ready** ? e.g. 1/1 nodes ready when kubeconfig exists
- **Open** ? Operations view for that cluster
- **Deploy** ? runs hypercluster deploy with live logs

Empty state:

When no clusters are registered, a glass panel prompts you to import cluster.conf or use the wizard.

4. Create Cluster Wizard

Five steps; generates cluster.conf under Application Support.

Step · Fields
1 ? Identity · Cluster name, distro (k3s / Kubespray), Kubernetes version
2 ? Nodes & SSH · Control plane IPs, worker IPs (tag input), SSH user, SSH key (browse)
3 ? Network · Pod CIDR, CNI plugin
4 ? Platform · MetalLB toggle + IP pool, KubeVirt + CDI toggle
5 ? Review · Full generated config preview

On step 5:

- **Deploy immediately after saving** ? runs deploy after registry write.
- **Save & Create** ? writes config and returns to dashboard (or starts deploy if checked).

Defaults match the k3s remote platform path (Cilium, cert-manager, KubeVirt v1.8.x pins in generated conf).

5. Day-2 Operations

Select a cluster from the dropdown, then run commands:

Button · hypercluster command
preflight · preflight
deploy · deploy
test · test
status · status
scale · scale
upgrade · upgrade
fetch-kubeconfig · fetch-kubeconfig
merge-kubeconfig · merge-kubeconfig
unlock · unlock
test --full · test --full
dry-run deploy · deploy -n
remove-node · remove-node <name> -f
Destroy cluster · destroy -f (type cluster name to confirm)
Reveal cluster.conf in Finder opens the config file location in macOS Finder.
Destroy requires typing the exact cluster name in the confirmation field.

6. Deploy & Log Console

When any command runs, the view switches to **Deploy** (log console):
- **Title** ? cluster name + command (+ dry-run suffix when applicable)
- **Stage card** ? parsed deploy stage (current/total + label) when hypercluster emits stage lines
- **Log console** ? streaming stdout/stderr (ANSI stripped for display)
- **Cancel job** ? sends SIGTERM to the running subprocess
On completion, a native **notification** shows success or failure.
The dashboard **Active job** stat reflects running state.

7. Settings

Control · Action
Custom hypercluster binary path · Override auto-detected binary
Save · Persists to registry settings
Import cluster.conf? · Same as dashboard import
Open data directory · Opens ~/Library/Application Support/Hypercluster in Finder

8. Command Palette

Open with **?K** (macOS) or **Ctrl+K**.
- Search filters clusters by name.
- Per cluster, quick actions: preflight, deploy, test, status, fetch-kubeconfig.
- **Escape** or **?K** again closes the palette.
Selecting a command starts the job and opens the log console.

9. Keyboard Shortcuts

Shortcut · Action
?K / **Ctrl+K** · Toggle command palette
Escape · Close command palette

10. Notifications & Tray

- **macOS menu bar tray** ? Open Hypercluster / Quit
- **Job notifications** ? title "Hypercluster", body includes command name and result
- **Tray click** ? show and focus main window

11. Data Storage

Path (macOS) · Content
~/Library/Application Support/Hypercluster/clusters.json · Registry + settings
~/Library/Application Support/Hypercluster/clusters/<name>/cluster.conf · Per-cluster config
Windows paths use the Tauri app data directory (see Settings ? Open data directory).

12. CLI Mapping Reference

The desktop invokes:

```
 exec "<hypercluster>" -c "<config_path>" [-n] [-v] [-f] <command> [args...]
```

JSON used internally (not shown in log view):

- Dashboard node counts ? status --json
- Future test result panels ? test --json

Human-readable test output appears in the log console; JSON mode keeps stdout clean for parsers.

Related documentation

- CLI guide
- Configuration reference
- Main README
- Documentation hub

Doc version: 2026-06-04T17:30:01Z · hypercluster v0.1.0